

ORIGINAL ARTICLE

Open Access



A cooperative jamming decision-making method based on multi-agent reinforcement learning

Bingchen Cai¹, Haoran Li¹, Naimin Zhang^{1*} , Mingyu Cao¹ and Han Yu¹

Abstract

Electromagnetic jamming is a critical countermeasure in defense interception scenarios. This paper addresses the complex electromagnetic game involving multiple active jammers and radar systems by proposing a multi-agent reinforcement learning-based cooperative jamming decision-making method (MA-CJD). The proposed approach achieves high-quality and efficient target allocation, jamming mode selection, and power control. Mathematical models for radar systems and active jamming are developed to represent a multi-jammer and multi-radar electromagnetic confrontation scenario. The cooperative jamming decision-making process is then modeled as a Markov game, where the QMix multi-agent reinforcement learning algorithm is innovatively applied to handle inter-jammer cooperation. To tackle the challenges of a parameterized action space, the MP-DQN network structure is adopted, forming the basis of the MA-CJD algorithm. Simulation experiments validate the effectiveness of the proposed MA-CJD algorithm. Results show that MA-CJD significantly reduces the time defense units are detected while minimizing jamming resource consumption. Compared with existing algorithms, MA-CJD achieves better solutions, demonstrating its superiority in cooperative jamming scenarios.

Keywords: Electromagnetic jamming, Multi-agent system, Reinforcement learning, Cooperative decision-making

1 Introduction

With the continuous advancement of jamming technologies and anti-jamming measures, the electromagnetic environment in defense and interception scenarios has become increasingly complex, making the jamming decision-making process highly challenging. Specifically, defenders often face cooperative detection from multiple adversarial radar systems. In such scenarios, deploying multiple active jammers for cooperative jamming has proven to be an effective approach to counter networked radar detection. Consequently, developing fast and efficient cooperative jamming decision-making strategies has become a research hotspot in the field of electronic countermeasures in recent years.

Traditional jamming decision-making methods can be broadly categorized into three types: (1) rule-based methods [1, 2], which construct predefined jamming rule libraries and select jamming patterns by matching real-time states with features in the library; (2) probabilistic reasoning methods [3, 4], which utilize probabilistic inference techniques, such as Bayesian networks, to make jamming decisions in specific confrontation scenarios; and (3) heuristic optimization algorithms [5, 6], which employ intelligent optimization techniques, such as tabu search and ant colony algorithms, to explore optimal jamming strategies. These methods heavily rely on the scale and quality of prior knowledge, yet in complex electromagnetic environments, jammers, as adversaries to radar systems, often struggle to obtain complete and accurate radar information for decision-making. Moreover, modern electronic countermeasure scenarios are highly dynamic, with rapid changes in the states of both radar and jamming systems.

* Correspondence: Naiminz@sohu.com

¹ Beijing Institute of Astronautical Systems Engineering, Beijing, 100076, China

Consequently, traditional static decision-making methods often yield suboptimal performance.

With the advent of the era of informationization and intelligence, artificial intelligence technologies, represented by reinforcement learning (RL), are driving transformative advancements in adversarial decision-making domains. Agents trained using RL algorithms have demonstrated exceptional performance in decision-making problems, including board games [7–9], multiplayer strategy games [10, 11], and military simulation confrontations [9, 12]. Inspired by these successes, numerous studies have attempted to apply RL to jamming decision-making. For example, [13] employed the dueling double deep Q-network algorithm to train agents for selecting jamming patterns, enabling the decision-making process to operate without any prior knowledge. Similarly, [14] applied a prioritized experience replay-based double deep Q-network to cooperative jamming decisions. This approach combines the states of multiple radars and jammers into a global state vector, using a centralized deep Q-network to estimate the global value of joint jamming actions, thereby determining both jamming targets and patterns. Furthermore, [15] utilized the double Q-learning algorithm to train jammers for jamming pattern selection and designed a heuristic reward function to accelerate training convergence. These studies effectively eliminate reliance on prior knowledge, addressing dynamic adversarial processes within a sequential decision-making framework. However, they primarily focus on selecting jamming patterns using discrete action spaces. In real-world adversarial scenarios, jamming patterns are often tightly coupled with jamming parameters, such as power and frequency, which significantly influence the effectiveness of a given jamming pattern. As a result, the joint optimization of jamming patterns and parameters has become a prominent research topic in recent years.

Several studies have attempted to jointly address jamming pattern selection and jamming parameter control in decision-making. For instance, [16] introduced heuristic-accelerated Q-learning into one-on-one jamming decision-making scenarios, where the jammer adaptively selects jamming patterns and power levels based on jamming effectiveness. Similarly, [17] modeled the tasks of jamming pattern selection and power control using both discrete and parameterized action spaces and applied hierarchical PPO and D3QN algorithms to solve the respective decision-making problems. Additionally, [18] proposed a hierarchical framework that utilizes a D3QN to select jamming patterns while constructing three policy networks to determine jamming power, timing, and duration, respectively. These policy networks were trained using the MADDPG algorithm. While existing studies have effectively addressed the joint decision-making of jamming patterns and parameters, most of these efforts focus solely on single-jammer scenarios. In multi-jammer

cooperative decision-making scenarios, single-agent reinforcement learning algorithms face challenges such as the curse of dimensionality and environmental instability [19]. These challenges hinder training convergence and result in suboptimal decision-making performance.

Multi-agent systems and multi-agent reinforcement learning (MARL) have been research hotspots in recent years [20–22]. Algorithms designed for multi-agent systems, such as QMix [23], MADDPG [24], and RODE [25], have demonstrated significant advantages in various cooperative decision-making tasks [26–28], effectively addressing issues like the curse of dimensionality and environmental instability to a certain extent. Among these, QMix is a representative algorithm that employs a centralized hyper-network during training to combine the action values of individual agents into a global value estimate. During execution, each agent independently computes its optimal action using its own DQN. QMix has shown outstanding performance in numerous cooperative multi-agent systems [29]. However, few studies have applied QMix and similar MARL algorithms to multi-dimensional joint decision-making in cooperative jamming scenarios.

This paper focuses on many-to-many electromagnetic jamming confrontation scenarios and proposes the MARL-based cooperative jamming decision-making (MA-CJD) algorithm. The decision-making tasks include selecting jamming targets, choosing jamming patterns, and controlling jamming power. The main contributions of this work are as follows:

- *Modeling the Cooperative Jamming Confrontation Scenario.* This paper considers a realistic many-to-many cooperative jamming decision-making scenario. Mathematical models for active suppression jamming, active deception jamming, and radar detection processes are constructed based on the jamming and radar equations. The confrontation process is modeled as a Markov game, where jamming targets, jamming patterns, and jamming power parameters are simultaneously included in the decision-making process. A jamming success probability is proposed as the reward function, and its computation method is provided.
- *Application of QMix Architecture.* The QMix multi-agent reinforcement learning algorithm is applied to the cooperative jamming scenario, enabling agents to effectively learn to collaborate with other jammers and achieve superior team performance. Adopting the QMix training framework, the proposed MA-CJD algorithm demonstrates superior performance in cooperative jamming decision-making scenarios, achieving significant improvements in resource efficiency and detection evasion rates compared to single-agent reinforcement learning algorithms.

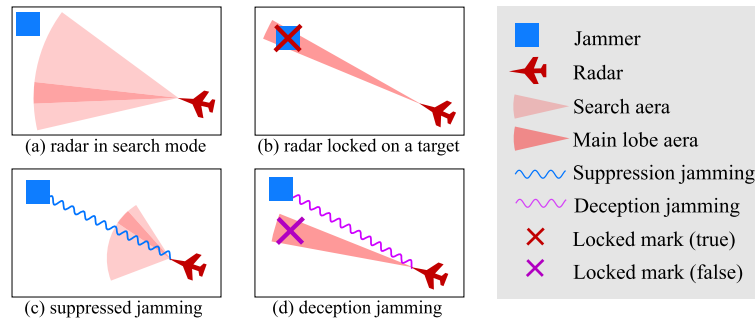


Figure 1 Cooperative jamming scenario

- *Introduction of Multi-Pass Deep Q-Networks (MP-DQN).* The MP-DQN network structure is introduced to handle parameterized action spaces, addressing challenges associated with joint optimization of discrete and continuous actions. As a key component of the MA-CJD algorithm, this innovation ensures robust learning efficiency and substantial performance improvements over baseline methods. Through a series of comparative experiments, the introduction of MP-DQN has been shown to significantly enhance the algorithm's capabilities. Specifically, compared to using the standard fully connected network structure in the QMix algorithm, the inclusion of MP-DQN reduces jamming resource consumption by 41.06% and decreases detection time by 42.30%.

The remainder of this paper is organized as follows: Sect. 2 provides a detailed description of the cooperative jamming confrontation scenario considered in this study and establishes the mathematical models for jamming and detection. Section 3 formulates the problem as a Markov game model, defining the state space, action space, and reward function for jamming decision-making. Section 4 introduces the neural network structure and loss function of the MARL-based cooperative jamming decision-making algorithm, as well as the training process. Section 5 details the simulation environment setup, hyperparameters for strategy optimization, and demonstrates the superiority of the MARL-based cooperative jamming decision-making algorithm through extensive comparative experiments. Finally, Sect. 6 summarizes the main contributions and findings of this study.

2 Cooperative jamming scenario

This paper considers an electromagnetic confrontation process in a defense interception scenario, where the defending side employs both active suppression jamming and active deception jamming to reduce the detection capability of the opponent's airborne radar. The confrontation

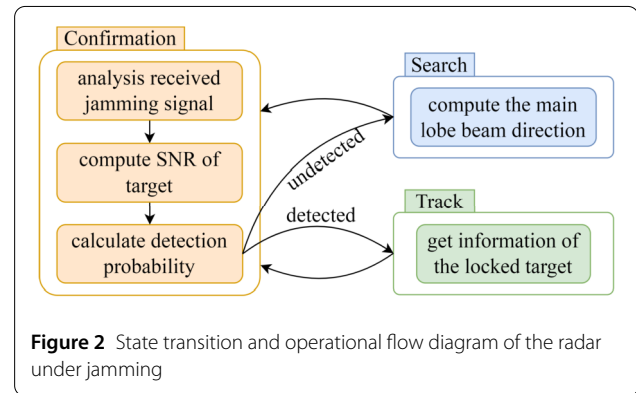


Figure 2 State transition and operational flow diagram of the radar under jamming

process between the radar and the jammer is illustrated in Fig. 1. Figure 1(a) shows a radar in search mode. When the radar detects the defender, it locks onto the target, as shown in Fig. 1(b). The defender can then use active suppression jamming to suppress the radar's detection range, as shown in Fig. 1(c), or employ active deception jamming to mislead the radar into locking onto a false target, as shown in Fig. 1(d). Our primary focus is on the process of strategy generation, and hence, the models constructed are at the functional level.

This study assumes that the radar detection process under jamming can be divided into three states: search, confirmation, and tracking. In the search state, the radar continuously updates the main lobe direction of its antenna beam according to predefined rules. If a true or false target exists within the main lobe's direction, the radar transitions to the confirmation state. In the confirmation state, the radar analyzes the received jamming signal power and target echo power to perform target detection, determining whether a target is detected. If the detection is successful, the radar transitions to the tracking state, where it locks onto the detected target. Otherwise, it reverts to the search state. The state transitions and operational flow of the radar under jamming are illustrated in Fig. 2.

The following sections provide a detailed explanation of the computational processes for the radar model and the

jammer model involved in the cooperative jamming scenario.

2.1 Radar model

In the search state, the main lobe beam direction of the radar antenna pattern periodically changes within a certain range. The main lobe beam direction θ_a at time t can be expressed as:

$$\theta_a = \begin{cases} n \times \theta_m, & t_c \in [0, \frac{1}{4}T_s], \\ \theta_{\max} - n \times \theta_m, & t_c \in (\frac{1}{4}T_s, \frac{1}{2}T_s], \\ -n \times \theta_m, & t_c \in (\frac{1}{2}T_s, \frac{3}{4}T_s], \\ -(\theta_{\max} - n \times \theta_m), & t_c \in (\frac{3}{4}T_s, T_s], \end{cases} \quad (1)$$

where θ_m represents the main lobe width, $\theta_{\max}$ denotes the scanning range, $n = \lfloor \text{mod}(t, T_s)/t_a \rfloor$ is the scan count, $\lfloor \cdot \rfloor$ represents the floor operation, $\text{mod}(\cdot)$ denotes the modulus operation, t_a is the pulse accumulation time, and $T_s = \theta_{\max}/\theta_m$ is the scanning period.

This study assumes that the radar focuses on targets located within its main lobe beam. According to the radar equation, the echo power of a true target is given by:

$$P_s = \frac{P_t G_t G_r \lambda^2 \sigma}{(4\pi)^3 R^4 L L_{\text{Atm}}}, \quad (2)$$

where P_t is the radar transmitter's peak power, G_t and G_r are the gains of the transmitting and receiving antennas, respectively, λ is the radar wavelength, σ represents the radar cross-section (RCS) of the target, R is the distance between the target and the radar, L denotes the radar system's total loss, and L_{Atm} accounts for atmospheric attenuation.

The radar model calculates the detection probability based on the signal-to-noise ratio (SNR) using Alberheim's approximate formula [30], as shown in Eq. (3).

$$\begin{aligned} pr_d(\text{SNR}) &= \frac{1}{1 + \exp(-B)}, \\ \text{where } B &= \frac{10^Z - A}{1.7 + 0.12A}, \\ Z &= \frac{\text{SNR} + 5 \times \lg M}{6.2 + 4.54/\sqrt{M} + 0.44}, \\ A &= \ln\left(\frac{0.62}{pr_{fa}}\right). \end{aligned} \quad (3)$$

For a true target, the SNR is defined as the ratio of the echo power to the sum of noise power and suppression jamming power. For a false target, the SNR corresponds to the ratio of deception jamming power to noise power. The calculation of SNR under jamming conditions is detailed in Subsect. 2.2.

In functional-level radar models, the target detection process is often simulated using Monte Carlo methods [31]. Specifically, the radar model generates a uniformly distributed random number d between 0 and 1. If $d \leq pr_d$, the radar is considered to have successfully detected the target; otherwise, if $d > pr_d$, the target is not detected. Among all the detected jammers and false targets, the one with the highest detection probability is selected for tracking. At this point, the radar transitions to the tracking state, calculating the azimuth, range, and velocity of the locked target relative to itself. These parameters are used to compute the detecting platform's heading angle, enabling it to adjust its trajectory toward the locked target and complete follow-up tasks.

2.2 Jamming model

This paper assumes that the jammer can employ two types of jamming methods: active suppression jamming and active deception jamming. Suppression jamming creates a background of interference in the radar receiver that masks the echo signals, making it difficult to detect true targets. Deception jamming generates false target echoes using signals with characteristics similar to the radar's, thereby misleading the radar. The received jamming signal power at the radar is calculated as follows:

$$P_{rj} = \frac{P_j G_j G_{rj} \lambda^2 B_r}{(4\pi R_j)^2 L_j L_r L_{\text{Atm}} B_j}, \quad (4)$$

where P_j is the jammer's transmission power, G_j is the jammer's transmitting antenna gain, G_{rj} is the radar's receiving antenna gain in the direction of the jammer, λ is the radar wavelength, R_j is the distance between the jammer and the radar, L_j is the jammer's overall transmission loss, L_r is the radar's overall receiving loss, L_{Atm} is the atmospheric loss, B_r is the instantaneous bandwidth of the radar receiver, and B_j is the bandwidth of the jamming signal.

Under stable external conditions, for a specific type of radar and jammer, it is assumed that G_j , λ , L_j , L_r , L_{Atm} , and B_j are constants. The radar's receiving antenna gain varies in different directions. Assume that the antenna gain within the main lobe direction is G_{rj1} , while the gain within the sidelobe direction is G_{rj2} . Under these conditions, the relationship between the received jamming signal power P_{rj} and P_j , R_j , and G_{rj} is expressed as shown in Eq. (5).

$$P_{rj} \propto \begin{cases} \frac{P_j G_{rj1}}{R_j^2}, & \text{if in the main lobe direction,} \\ \frac{P_j G_{rj2}}{R_j^2}, & \text{if in the sidelobe direction.} \end{cases} \quad (5)$$

In the subsequent discussion, P_{rjd} represents the deception jamming power received by the radar, and P_{rjs} represents the suppression jamming power received by the radar.

This study assumes that the radar is equipped with anti-jamming measures, represented by the anti-jamming factor D . The jamming signal power after applying anti-jamming measures is expressed as:

$$\bar{P}_{rj} = D \cdot P_{rj}, \quad (6)$$

where, D is related to the direction of the incoming jamming signal. Jamming signals received from the sidelobes are more easily suppressed compared to those received from the main lobe, as more anti-jamming measures are available for sidelobe signals. Additionally, D also depends on the radar system design, as different radar types are equipped with varying levels of anti-jamming capabilities.

Furthermore, the radar can identify false targets generated by deception jamming based on the characteristics of the jamming signal. When the jamming-to-noise ratio (JNR) is sufficiently high, the signal characteristics become more pronounced. This leads to higher correlation among false target signals and lower correlation between false and real targets, thereby increasing the probability of identifying false targets [32–34]. Based on this, it is assumed that the relationship between the JNR of deception jamming and the probability of identifying false targets, pr_{ij} , is calculated as follows:

$$pr_{ij} = \frac{1}{1 + \exp(w \cdot (\text{JNR}_{0.5} - \text{JNR}))}. \quad (7)$$

In this equation, w represents the gradient of the identification probability with respect to the JNR, and $\text{JNR}_{0.5}$ denotes the JNR value when the identification probability is 0.5. Assuming the power of deception jamming signals received by the radar is P_{rjd} , and the radar noise power is P_n , the JNR is given by $\text{JNR} = P_{rjd}/P_n$. The success of identifying jamming signals can then be determined using Eq. (7).

Deception jamming signals that are not identified successfully will create a false target within the radar's main beam coverage. The SNR of the false target is calculated as:

$$\text{SNR}_f = \frac{DP_{rjd}}{P_n}. \quad (8)$$

The SNR of the real target is influenced by suppression jamming. Let the total power of suppression jamming signals received by the radar be P_{rjs} , and the radar's pulse compression accumulation gain be G_a . The SNR of the real target within the main beam under the influence of suppression jamming is calculated as follows:

$$\text{SNR}_a = \frac{G_a P_s}{DP_{rjs} + P_n}. \quad (9)$$

By substituting SNR_f and SNR_a into Eq. (3), the detection probabilities can be calculated, enabling the determination of target detection under jamming conditions.

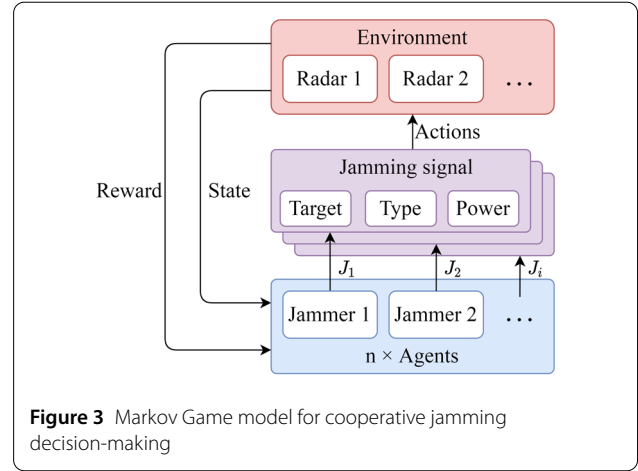


Figure 3 Markov Game model for cooperative jamming decision-making

3 Markov game model

The cooperative jamming scenario considered in this paper is a multi-agent decision-making process involving multiple jammers and radar systems, categorized as a multi-agent game. The entire confrontation process can be described using a Markov game model. The Markov game model is defined by a six-tuple [35]: $(\mathcal{N}, \mathcal{S}, (\mathcal{A}^i)_{i \in \mathcal{N}}, p, r, \gamma)$. Here, $\mathcal{N}$ represents the set of agents, $\mathcal{S}$ is the state space, $\mathcal{A}^i$ denotes the action space of agent $i \in \mathcal{N}$, p is the state transition function, r is the reward function, and γ is the discount factor.

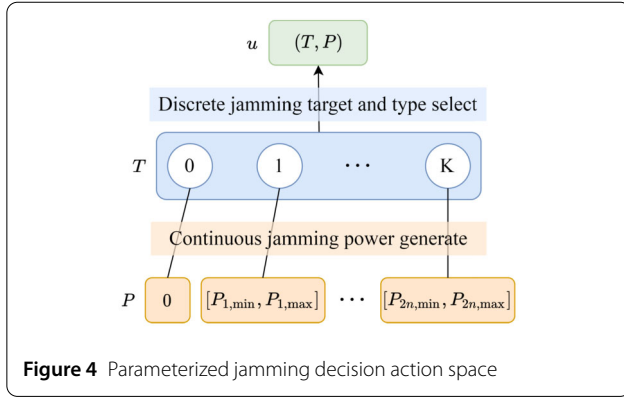
In the jamming decision-making process considered, $\mathcal{N}$ refers to the set of all jammers, where each jammer acts as an agent that autonomously makes decisions. The radar model serves as the environment that implements state transitions. In the following, $\mathcal{J}$ represents the set of jammers and $\mathcal{R}$ denotes the set of radar systems. The overall structure of the Markov Game model is illustrated in Fig. 3. The design of the actions, rewards, and states is detailed below.

3.1 Action space

In the cooperative jamming decision-making process, each jammer must decide its target, jamming type, and power level at each step. This paper designs the jamming strategy as a parameterized action space consisting of both discrete and continuous actions. The schematic of the action space is shown in Fig. 4.

The jamming target and type are represented as discrete actions, while the jamming power level is represented as a continuous action. The action u^i for jammer i is denoted by the tuple (T^i, P^i) .

Here, T^i is a discrete integer value representing the jamming target and type. When $T^i = 0$, the jammer performs no jamming action. If $\text{mod}(T^i, 2) = 0$, the jamming type is deception jamming; if $\text{mod}(T^i, 2) = 1$, the type is suppression jamming. The expression $\lfloor (T^i + 1)/2 \rfloor$ indicates the target ID of the jamming action. The maximum value of



T^i is denoted as K , where $K = 2|\mathcal{R}|$ ($|\mathcal{R}|$ is the number of radar systems).

$P^i \in [0, 1]$ is a continuous value representing the jamming power level. Let $P_{j,\max}$ and $P_{j,\min}$ denote the maximum and minimum transmission power of the jammer, respectively. The relationship between the actual jamming power P_j and the jamming power level P^i is defined as:

$$P_j = P^i \cdot (P_{j,\max} - P_{j,\min}) + P_{j,\min}. \quad (10)$$

3.2 Reward function

The objective of cooperative jamming decision-making is to minimize radar detection capabilities with the least amount of jamming resources, aiming to reduce the likelihood of radar successfully tracking real targets. Therefore, the reward function should effectively reflect this optimization goal. In this paper, the reward function is designed with three components: tracking penalty r_d , resource consumption penalty r_p , and jamming success probability reward r_j . The reward function is computed as follows:

$$r = r_d + r_p + r_j. \quad (11)$$

r_d represents the penalty applied when a radar detects and successfully locks onto a defense unit. The value of r_d depends on the radar type: radars with higher threat levels result in a larger absolute values of r_d . This penalty encourages the agent to prioritize allocating more jamming resources to radars with higher threat levels.

r_p is a penalty term linearly related to the jamming power, designed to incentivize agents to conserve jamming resources. Let the minimum and maximum values of r_p be $r_{p,\min}$ and $r_{p,\max}$, respectively. The calculation of r_p is as follows:

$$r_p = -r_{p,\max} - (r_{p,\max} - r_{p,\min}) \frac{P_j - P_{j,\min}}{P_{j,\max} - P_{j,\min}}. \quad (12)$$

In this study, $r_{p,\min}$ is set to -0.1 , and $r_{p,\max}$ is set to -0.01 .

To assist the agent in identifying nuanced state information and to avoid sparse rewards that may lead to ineffective exploration, a jamming success probability reward r_j is proposed. A higher value of r_j indicates a greater likelihood of successful jamming.

For deceptive jamming, the success probability corresponds to the radar tracking a false target. Let the maximum SNR value among all real targets detected by the radar at time t be $\text{SNR}_{\max}$. Define the set of false targets with $\text{SNR}_f > \text{SNR}_{\max}$ as $\mathcal{F}$. It is assumed that if any false target in $\mathcal{F}$ passes detection, the radar is highly likely to track the false target. The success probability reward for deceptive jamming is computed as shown in Eq. (13):

$$r_j = 1 - \prod_{i \in \mathcal{F}} (1 - pr_{d,i}), \quad (13)$$

where, $pr_{d,i}$ is the detection probability of a false target $i \in \mathcal{F}$, calculated using Eq. (3).

For suppressive jamming, its effect is to reduce the SNR of target echoes, thereby lowering their detection probability. The reduction in detection probability is defined as the jamming success probability reward, which is calculated as:

$$r_j = pr_d \left(\frac{G_a P_s}{DP_{rjs} + P_n} \right) - pr_d \left(\frac{G_a P_s}{P_n} \right). \quad (14)$$

In Eq. (11), r_d is a manually defined Hyperparameter ranging from -1.2 to -0.8 , emphasizing its critical role in reducing radar detection capabilities. r_p is scaled linearly based on power levels, with values ranging from -0.1 to -0.01 , ensuring its magnitude remains smaller than r_d to reflect its secondary importance while preventing the agent from learning overly conservative strategies. r_j , as an expected probability value distributed between 0 and 1, dynamically adjusts to approach the magnitude of r_d when potential jamming rewards are higher, encouraging interference actions. This design prioritizes reducing radar detection while balancing power efficiency and promoting proactive jamming under favorable conditions.

3.3 State space

The environment's state serves as the decision-making basis for the jammer. In the cooperative jamming decision task, the state space is divided into two parts: inherent attributes and dynamic features.

The inherent attributes mainly include the radar's detection capability and anti-jamming capability. The radar's detection capability is represented by the radar's peak transmit power P_t , main beam width θ_m , and search scan period T_s . The radar's anti-jamming capability is represented by the radar type Type_r . These inherent attributes reflect the threat level of the radar and are crucial for determining the jamming targets.

Dynamic features include the radar's main beam direction θ_a , the radar's position pos_r , and the jammer's position pos_j . These features indicate the relative distance and orientation between the jammer and the radar, which significantly impact the effectiveness of the jamming.

In summary, the state vector for cooperative jamming decision-making process is represented as:

$$s = \left[(P_t^i, \theta_m^i, T_s^i, \text{Type}_r^i, \theta_a^i, \text{pos}_r^i)_{i \in \mathcal{R}}, (\text{pos}_j^j)_{j \in \mathcal{J}} \right]. \quad (15)$$

4 MARL-based cooperative jamming decision-making

The cooperative jamming decision problem considered in this paper faces two main challenges: First, the decision-making process is a multi-agent game scenario involving multiple jammers, making algorithms designed for single agents unsuitable. Second, the joint optimization of jamming modes and parameters results in a parameterized action space, requiring special design of the neural network architecture. To address these challenges, this paper innovatively introduces the QMix multi-agent reinforcement learning algorithm, and integrates the MP-DQN network structure to handle the parameterized action space. An MARL-based cooperative jamming decision-making (MA-CJD) algorithm is proposed. Furthermore, the double DQN mechanism is employed to further improve the decision-making performance of the agents.

4.1 Training framework

Compared to single-agent reinforcement learning, multi-agent reinforcement learning faces two main challenges: (1) the instability of the environment during training. Each agent is part of the environment for the other agents. Its strategy updates will affect the convergence of the strategies of other agents. (2) the rewards can be deceptive, as the global reward sometimes misguides a specific agent, leading to a phenomenon known as agent "laziness" [36].

This paper adopts a centralized training and decentralized execution architecture, introducing the QMix algorithm. By utilizing QMix-based credibility allocation techniques, the aforementioned issues are effectively alleviated. The QMix algorithm is a value-based reinforcement learning method where each jammer agent has its own value network to estimate the action values. Additionally, there is a Mixing hyper-network $Q_{\text{total}}(s_t, Q^1, \dots, Q^N; \omega_{\text{mix}})$ during training. In the training process, the Mixing network encodes state information into its parameters. The network input consists of the predicted action values from each agent for its own actions, while the output is the predicted global action value. The parameters of the Mixing super-network are non-negative, ensuring that the global action value is optimal while the individual action values are also optimized. The trained Mixing network can reasonably allocate the global value to each jammer agent,

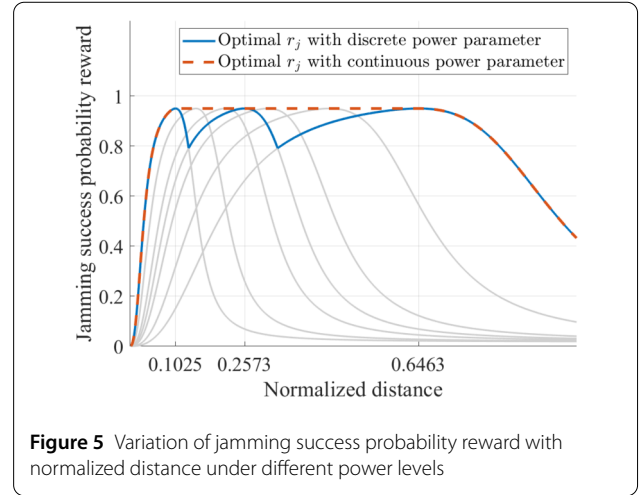


Figure 5 Variation of jamming success probability reward with normalized distance under different power levels

helping each agent correctly learn the contribution of its actions to the overall team.

4.2 Network structure

The action space designed above is a parameterized action space, which includes both discrete actions and the continuous action parameters corresponding to each discrete action. Traditional DQN network structures are not capable of simultaneously outputting value estimates for both discrete and continuous actions. A common approach is to discretize the continuous action parameters, transforming the action space into a discrete one. However, this leads to a loss in learning precision.

As an example of deception jamming, when the relative distance between the jammer and the radar changes, the variation curves of the jamming success probability reward r_j under different power parameters are shown in Fig. 5. The distance in Fig. 5 has been normalized.

The gray curves in the figure represent the variation of the jamming success probability reward with the relative distance for power levels of 0.0, 0.2, 0.4, 0.5, 0.6, 0.8, and 1.0. If the power levels are discretized, the jammer can only choose power levels of 0.0, 0.5, and 1.0, and the optimal r_j obtainable under this condition as the relative distance changes is shown by the blue curve. In contrast, with continuous power parameters, the optimal r_j as a function of the relative distance is represented by the red curve. From the figure, it can be seen that the discretized power parameters can only achieve the optimal result at the normalized relative distances of 0.1025, 0.2573, and 0.6463. In all other relative distance cases, there are always other power parameters that yield a higher jamming success probability reward.

In order to avoid discretizing the power level, this paper proposes the use of the MP-DQN structure. MP-DQN consists of two components: the Actor network and the multi-passed Q network. The Actor network generates the

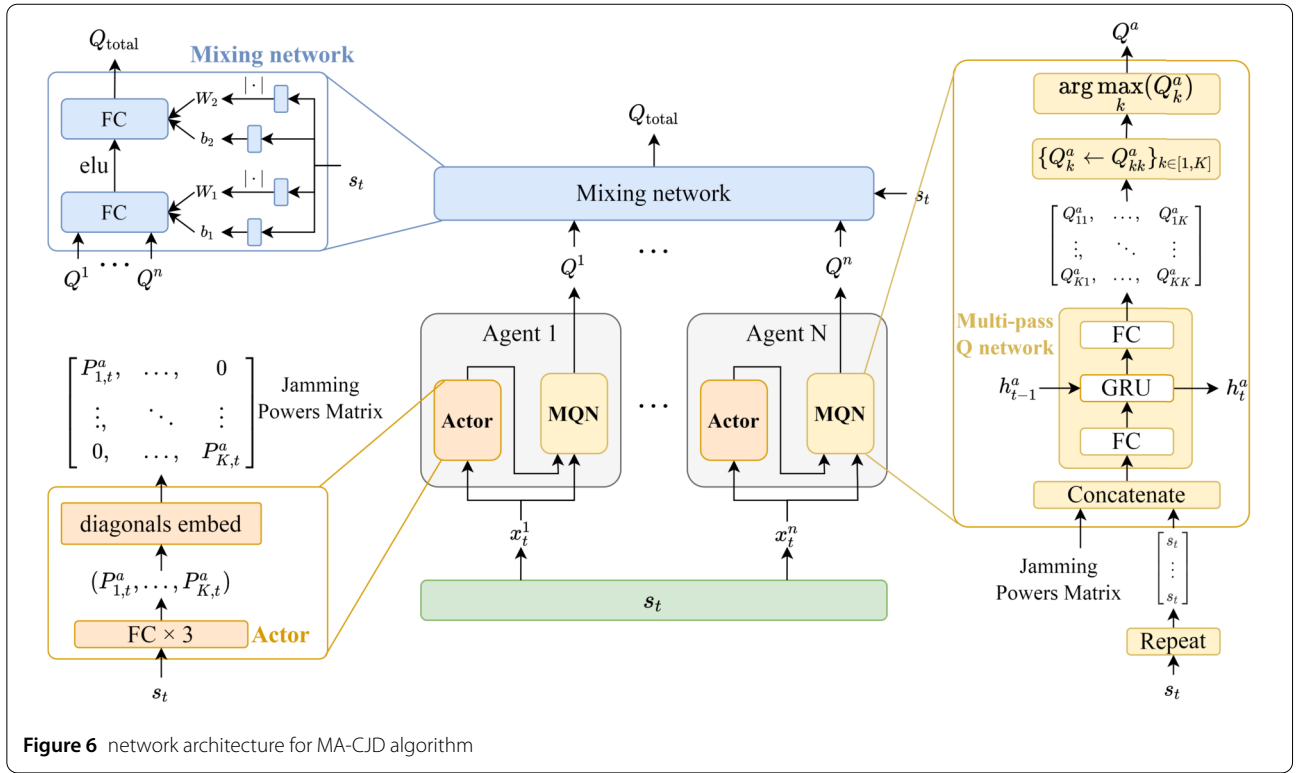


Figure 6 network architecture for MA-CJD algorithm

corresponding power level P^i for each discrete action T^i , and both the power level and the state vector are input into the Q network, which computes the action value estimates for each action. This network structure enables the agent to compute the corresponding continuous power level while selecting discrete actions. As a result, the jammer can use any power level within the range of $[0, 1]$ to achieve a higher jamming success probability, as shown by the red curve in Fig. 5.

The MP-DQN network is integrated into the QMix architecture, as shown in Fig. 6. Each agent network consists of two parts: the Actor network and the Q network. The Actor network is represented as $\mu^a(s_t; \theta)$. The Q network is represented as $Q^a(s_t, T^a_t, \mu^a; \omega)$.

In the figure, the Actor network uses a three-layer fully connected neural network to generate the corresponding power level for each discrete action based on the input state vector. The power levels are concatenated with the state variables and input into the Q network through different channels. The hidden layers of the Q network employ GRU units, enabling the agent to consider past actions and states during decision-making. The action is selected using an ε -greedy strategy based on the output value estimates, and the corresponding action values are input into the Mixing network to compute the global value estimate. The Mixing network is represented as $Q^a_{mix}(s_t, \{Q^a\}_{a \in \mathcal{J}}; \omega_{mix})$.

The integration of MP-DQN into the QMix architecture addressed the challenge of handling parameterized action spaces in multi-agent jamming decision-making. Specifically, the discrete actions (i.e., target and mode selection T^i) and continuous actions (i.e., power level P^i) are jointly optimized through a unified framework. The Actor network generates continuous power levels for each discrete action, while the Q network evaluates the action-value function by considering both the discrete action and its associated continuous parameters. This dual-network structure ensures that the agents can make precise decisions in a hybrid action space without the need for discretization, thereby improving the adaptability and effectiveness of the jamming strategy.

The key innovation of the proposed architecture lies in its ability to seamlessly combine centralized training with decentralized execution in a parameterized action space. As illustrated in Fig. 6, the Mixing network aggregates the individual action-value estimates from all agents while accounting for the continuous power levels generated by the Actor networks. This enables the system to achieve accurate global value decomposition and credit assignment, even in complex scenarios where both discrete and continuous actions are involved. Furthermore, the use of GRU units in the Q network allows the agents to incorporate historical state-action information into their decision-making process, enhancing the robustness of the jamming strategy in dynamic environments.

4.3 Parameter optimization

The TD algorithm is used to train the parameters of the Q network and the Mixing network. Target networks are used to compute the TD targets, where the Q network and Mixing network structures of the target networks are the same as the original networks, with parameters denoted as $\tilde{\omega}$ and $\tilde{\omega}_{\text{mix}}$, respectively. The loss function is calculated as follows:

$$\mathcal{L}(\omega, \omega_{\text{mix}}) = (y_t - \hat{y}_t)^2, \quad (16)$$

$$\text{where } \hat{y}_t = Q_{\text{mix}}\left(s_t, \{Q^a\}_{a \in \mathcal{J}}; \omega_{\text{mix}}\right), \quad (17)$$

$$y_t = r_t + \gamma \cdot Q_{\text{mix}}\left(s_{t+1}, \{\tilde{Q}^a\}_{a \in \mathcal{J}}; \tilde{\omega}_{\text{mix}}\right), \quad (18)$$

$$\tilde{Q}^a = \max_{k \in [0, K]} Q^a(s_t, k, \mu^a; \tilde{\omega}). \quad (19)$$

The training objective of the Actor network is to maximize the action value estimates. First, the sum of the action values output by each agent's Q network is calculated:

$$Q_{\text{sum}}^a = \sum_{k=1}^K Q^a(s_t, k, \mu^a(s_t; \theta); \omega). \quad (20)$$

Next, the sum of all agents' action values is fed into the Mixing network to obtain the global action value sum:

$$\hat{Q} = Q_{\text{mix}}\left(s_t, \{Q_{\text{sum}}^a\}_{a \in \mathcal{J}}; \omega_{\text{mix}}\right). \quad (21)$$

Finally, the gradient of the Actor network's parameters is computed as $\nabla_{\theta}(-\hat{Q})$.

To improve the decision-making performance and training speed of the agents, Double DQN mechanism [37] is introduced during training. Double DQN improves the calculation of the TD target y_t in Eq. (16). The computation of y_t is divided into two steps: selection and evaluation. First, the action value is computed using the original network, and the action with the maximum value is selected:

$$T^{a*} = \arg \max_{k \in [0, K]} Q^a(s_t, k, \mu^a; \omega). \quad (22)$$

Then, the action value of T^a at the next time step is computed using the target network, and the TD target is further obtained:

$$y_t = r_t + \gamma \cdot Q_{\text{mix}}\left(s_{t+1}, \{Q^a(s_t, T^{a*}, \mu^a; \tilde{\omega})\}_{a \in \mathcal{J}}; \tilde{\omega}_{\text{mix}}\right). \quad (23)$$

In the original QMix algorithm training process, both selection and evaluation are performed using the target network. In contrast, Double DQN uses the original network

Table 1 Parameters for different types of radar

Parameter	1	2	3	4
Transmitting Power P_t	300 MW	300 MW	180 MW	100 MW
Main-Lobe Anti-Jamming Factor D_m	-5 dB	-5 dB	-10 dB	-10 dB
Side-Lobe Anti-Jamming Factor D_s	-15 dB	-10 dB	-20 dB	-15 dB
Detection Penalty r_d	-0.9	-1.1	-1.0	-1.0

for selection and the target network for evaluation, resulting in a smaller estimate of the TD target. This effectively alleviates the overestimation issue caused by maximization and bootstrapping when computing the TD target.

Let $Q(s_t, T_t^a, \mu^a; \omega_{\text{mix}}, \omega)$ represent the overall structure composed of the Q network and the Mixing network. The training process of the MA-CJD with Double DQN is shown in Algo. 1.

5 Simulation experiments

This chapter designs a many-to-many cooperative jamming confrontation scenario. The jamming decision agents are trained using the MA-CJD algorithm.

5.1 Scenario setup

The simulation scenario designed in this paper includes the following settings:

- 1) The scenario includes four types of radar targets. Each type has distinct anti-jamming capabilities, detection capabilities, and threat levels. The key parameters for each type of radar are set based on [17], as shown in Table 1.
- 2) All jammers are assumed to have a RCS of $\sigma = 1 \text{ m}^2$, a radar system's total loss of $L = 10 \text{ dB}$, atmospheric attenuation of $L_{\text{Atm}} = 2 \text{ dB}$, and noise power of $P_n = 3 \text{ dB}$.
- 3) The scenario contains four jammers, each located 50 km from the center of the map at azimuth angles of 45° , -45° , 135° , and -135° relative to the center.
- 4) The scenario includes four radar targets. At the start of each round, the distances of the radar targets from the map center are randomly generated based on a normal distribution with a mean of 400 km and a standard deviation of 30 km. The azimuth angles are randomly generated based on a uniform distribution in $(-180^\circ, 180^\circ]$.
- 5) To simplify the simulation, only motion in the top-down plane is considered, and the altitude of the entities is ignored. All targets move in straight lines toward the map center while searching. Once a target locks onto a defense unit or a decoy, it switches to tracking mode and adjusts its velocity direction accordingly.

The scenario includes 4 jammers and 4 radar targets. Therefore, the number of agents is 4, and the maximum

Algorithm 1 MA-CJD with Double DQN

Require: Set of jammer agents $\mathcal{J}$, set of radar agents $\mathcal{R}$, state space dimension d_s , exploration parameter ε , learning rates $\{\alpha_t, \beta_t\}_{t>0}$, batch size B , maximum number of training steps MAX_STEP

Ensure: Neural network parameters of jammer agents $\omega, \omega_{\text{mix}}, \theta$

- 1: Randomly initialize the parameters of the original network $\omega_1, \omega_{\text{mix},1}, \theta_1$, and initialize the target network parameters $\tilde{\omega}_1 = \omega_1, \tilde{\omega}_{\text{mix},1} = \omega_{\text{mix},1}, \tilde{\theta}_1 = \theta_1$
- 2: Set the number of discrete jamming actions $K = 2 \times |\mathcal{R}| + 1$
- 3: **for** $\text{step} = 1, \dots, \text{MAX_STEP}$ **do**
- 4: Set the GRU unit hidden states of both the original and target networks as zero vectors of size $|\mathcal{J}| \times d_s$.
- 5: Initialize P_0 as a zero vector of size $|\mathcal{J}| \times K$, and T_0 as a zero vector of size $|\mathcal{J}| \times 1$
- 6: **for** $t = 1, \dots, T$ **do**
- 7: Compute the jamming power matrix $x = \mu(s_t; \theta_t)$
- 8: Choose the discrete jamming action and power (T_t, P_t) according to the ε -greedy strategy:

$$(T_t, P_t) = \begin{cases} \text{uniform random sampling} & \text{with probability } \varepsilon \\ (k, x(k)) \text{ where } k = \arg \max_{k \in [0, K]} Q(s_t, k, x(k); \omega_{\text{mix},t}, \omega_t) & \text{with probability } 1 - \varepsilon \end{cases} \quad (24)$$

- 9: Get r_t and s_{t+1} , and store $[s_t, T_t, P_t, r_t, s_{t+1}]$ in the experience replay buffer
- 10: **end for**
- 11: Randomly sample B episodes from the experience replay buffer
- 12: **for all** $b \in [0, B]$ **do**
- 13: Initialize the GRU unit hidden states of both the original and target networks as zero vectors
- 14: **for** $t = 1, \dots, T$ **do**
- 15: $q_t = Q(s_t, T_t, P_t, \omega_{\text{mix},t}, \omega_t)$
- 16: Compute the TD target y_t using Eq. (23)
- 17: Compute the Actor network optimization target $\hat{Q}_t$ using Eq. (21)
- 18: Update network parameters:

$$\begin{aligned} \omega_{t+1} &\leftarrow \omega_t - \alpha_t \nabla_{\omega} (y_t - q_t)^2 \\ \omega_{\text{mix},t+1} &\leftarrow \omega_{\text{mix},t} - \alpha_t \nabla_{\omega_{\text{mix}}} (y_t - q_t)^2 \\ \theta_{t+1} &\leftarrow \theta_t + \beta_t \nabla_{\theta} \hat{Q}_t \end{aligned} \quad (25)$$

- 19: **end for**
- 20: **end for**
- 21: **end for**

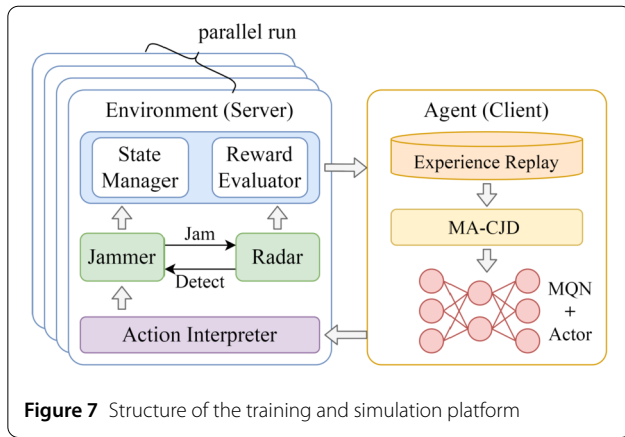
value for discrete actions T^i is $K = 9$. Radar types are represented using one-hot encoding, and positions are represented as 2D vectors. Thus, the state vector dimension is 48.

5.2 Implementation details and hyperparameter settings

To enable efficient training and execution of cooperative jamming decision agents, this paper designs a reinforcement learning training system based on a Client-Server architecture. The simulation environment server is implemented in C++, which includes the jammer model, radar model, and interaction logic among entities. The server provides interfaces for environment creation, reset, and step execution.

The client is built using Python 3.12 and implements experience pool management, agent training, and decision-making processes. Interactions between agents and the environment are facilitated through the simulation environment server's APIs. Multiple independent environment instances run in parallel on the server, significantly improving the efficiency of sample collection by agents. The architecture of the training and simulation platform used in the experiments is illustrated in Fig. 7.

We built the agents' Q network, Actor network, and Mixing network using PyTorch 2.4 in a CUDA 12.4 environment. The Q network features an input layer with 128 neurons connected through a linear layer activated by the ReLU function. Its hidden layer consists of 128-dimensional GRU units, and the output layer comprises a



linear layer with 9 neurons, matching the number of discrete actions.

The Actor network is a three-layer fully connected network, with input and hidden layer dimensions of 128, both employing ReLU activation functions. The output vector is mapped to the range $[0, 1]$ using a Sigmoid function to adhere to the constraints of jamming power levels. The Mixing network comprises a two-layer hyper-network utilizing ELU activation functions. The first layer has 64 dimensions, while the output layer consists of a single neuron. The weights and biases of the two-layer network are generated by linear layers with 128 dimensions, and the weights are constrained to their absolute values to satisfy the Individual-Global-Max condition, as proposed in [38].

During training, agents collect experiences using the ϵ -greedy strategy. In our experimental setup, the initial value of ϵ is configured as 0.95, enabling the agents to extensively explore the action space with a high degree of randomness during the initial phases of training. As the training process advances, ϵ undergoes a gradual decay, reducing to a final value of 0.05 over the course of 100,000 training steps. Beyond this point, ϵ remains fixed at 0.05, thereby shifting the emphasis from exploration to exploitation of the learned policy. This decay mechanism is strategically designed to ensure thorough exploration in the early stages of training, while progressively transitioning to a more exploitation-oriented approach in later stages. Such a design effectively balances the trade-off between discovering novel strategies and refining existing ones, ultimately facilitating the convergence to an optimal policy.

For the Q network and Mixing network, the learning rate α is set to 0.005, whereas for the Actor network, the learning rate β is set to 0.003. All networks use the Adam optimizer to improve the gradient descent process.

In each training cycle, agents interact with the environment for 16 episodes, storing the experiences in a replay buffer. Subsequently, 4 training iterations are conducted, each sampling 32 episodes of experience data for learning.

5.3 Experimental results and analysis

Under the aforementioned environment and parameter settings, four training sessions were conducted on hardware equipped with an Intel Core i9-13900 CPU and Nvidia GeForce 4090 GPU, with each session lasting 100,000 steps. During training, evaluations were performed every 100 steps. For evaluation, the agents always selected the actions with the highest value. Two evaluation metrics were tracked: *total episode reward* and *total radar lock time on the defense units per episode*. Each evaluation involved 32 episodes, and the average of the evaluation metrics was calculated. The resulting curves of the loss function and the two evaluation metrics during training are shown as blue curves in Fig. 8.

In Fig. 8, the curves represent the mean values across the 4 training sessions, with the shaded areas indicating the standard deviation. The experimental results demonstrate that after training with the MA-CJD algorithm, the total radar lock time on the defense units decreased by over 60% compared to the beginning of training. This indicates that the agents have acquired effective cooperative jamming decision-making capabilities, as a shorter lock time suggests more effective jamming.

To prove the superiority of the MP-DQN structure introduced in this study for handling the parameterized action space, this study compared it to a discretized approach. For agent $i \in \mathcal{J}$, the jamming power P^i was restricted to the discrete set $\{0, 0.5, 1\}$. This resulted in a 27-dimensional discrete action space: $\{0, 0.5, 1\} \times \{i | i \in \mathbb{Z}, 0 \leq i \leq 8\}$. In this setup, agents did not use the Actor network but instead relied on DQN to generate action values for all 27 actions. With the training settings as described above, the agents were trained using the QMix algorithm and the PER-DDQN cooperative jamming decision-making algorithm proposed in [14]. The loss function and evaluation metric curves are shown as the red curve in Fig. 8. It is evident that the introduction of MP-DQN significantly improved the agents' decision-making performance after convergence. This improvement is attributed to the limitations of discretizing continuous action parameters, which sacrifices precision. In certain scenarios, agents are unable to select the optimal jamming power, leading to suboptimal performance.

To validate the effectiveness of the double DQN mechanism introduced in this study, training was conducted using the MA-CJD algorithm without double DQN. The results are shown as the green curves in Fig. 8. The curves illustrate that, compared to the algorithm without double DQN, the one incorporating double DQN enables the agents to converge faster, with lower variance during the training process and higher average rewards after training. These experimental results demonstrate that in the multi-jammer cooperative jamming scenarios, the double DQN algorithm effectively mitigates the overestimation of the

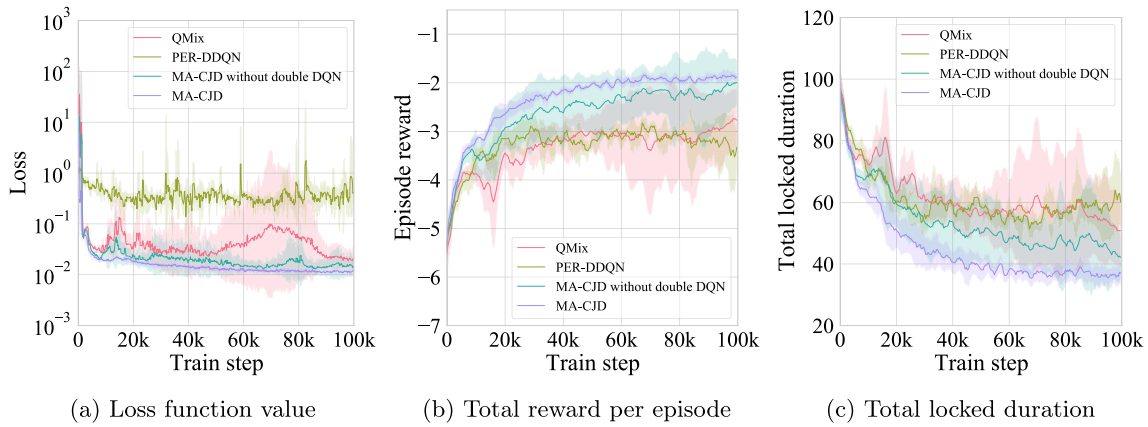


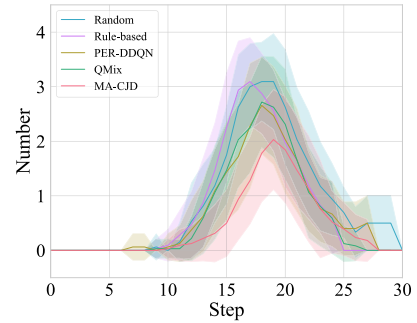
Figure 8 Training process of MA-CJD, QMix, PER-DDQN, and MA-CJD without Double DQN

TD targets. This leads to a more accurate learned action-value function for the agents, enhancing their decision-making performance.

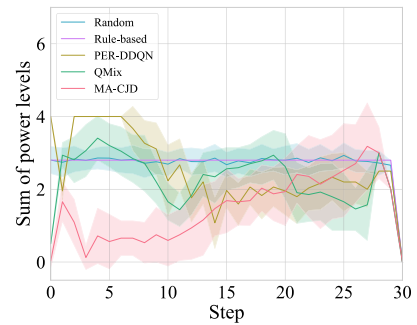
The superiority of the proposed MA-CJD algorithm was further validated by comparing the jamming strategies trained using MA-CJD against those produced by the QMix, the PER-DDQN, rule-based policy, and random policy. Each decision-making algorithm was evaluated through 32 experiments in randomly generated scenarios.

For each step, the number of radars successfully locking onto defense units was recorded. A lower count of locked radars indicates more effective jamming. The statistical results are shown in Fig. 9(a). The curves in the figure represent the average number of radars locking onto defense units at each step, with the shaded areas indicating the standard deviation. The comparison reveals that the MA-CJD strategy consistently results in fewer radars locking, demonstrating its superior jamming decision-making effectiveness.

The total sum of the jamming power levels P^i for all jammers at each step was recorded as well. A higher total P^i reflects greater utilization of jamming power resources at that moment. Figure 9(b) illustrates these results. The figure shows that the MA-CJD algorithm intelligently adjusts the jamming power parameters according to the state. At the beginning of the episodes, when the distance between radars and jammers is relatively large, radars have difficulty detecting the defense units, thus requiring minimal jamming power. However, as the distance decreases, higher jamming power is needed to counter radar detection. Among all tested methods, only MA-CJD demonstrated the capability to learn state-dependent adaptive adjustments of jamming power effectively.



(a) Number of radars locking onto defense units

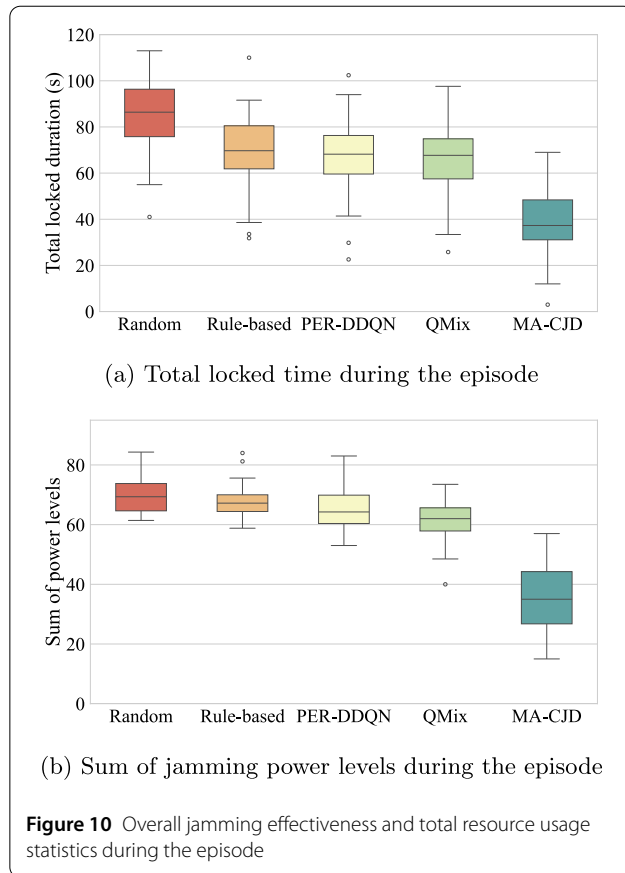


(b) Sum of jamming power levels

Figure 9 Changes in resource utilization and jamming effectiveness during each step of the episodes

These results highlight the efficiency and adaptability of the MA-CJD algorithm in resource allocation and jamming effectiveness.

The total radar-lock duration and the total sum of jamming power levels P^i over an entire episode were also analyzed. The results are displayed in Fig. 10(a) and Fig. 10(b), respectively. Each box plot represents the distribution of



results obtained across 32 repeated experiments for each algorithm. The edges of each box denote the first and third quartiles of the data, while the solid line inside the box indicates the median.

Figure 10(a) shows the total duration that defense units were locked onto by radars over an episode. The median lock time for the MA-CJD algorithm is significantly lower than that of the other algorithms, demonstrating its superior ability to evade radar detection. Figure 10(b) illustrates the total sum of the jamming power levels P^i for each algorithm over an episode. Again, the median value for MA-CJD is notably lower than those of other algorithms, indicating its ability to achieve effective jamming with reduced resource usage. These results reinforce the efficiency of the MA-CJD algorithm in minimizing radar detection time while also conserving jamming power resources, showcasing its practical advantages in cooperative jamming scenarios.

Table 2 summarizes the average radar-lock duration and the total jamming power levels P^i across multiple experiments for different jamming strategies. The results demonstrate the significant advantages of the proposed MA-CJD algorithm over other strategies. MA-CJD achieves the shortest average radar lock duration at 37.91 seconds, outperforming rule-based policy by 45.98%. The

Table 2 Total locked duration and sum of jamming power levels for different strategies

	Random	Rule-based	PER-DDQN	QMix	MA-CJD
Total locked duration	84.49 s	70.18 s	66.32 s	65.70 s	37.91 s
Sum of jamming power levels	69.61	67.38	64.89	60.98	35.94

total jamming power parameter for MA-CJD is 35.94, which is a 46.66% reduction compared to rule-based policy.

6 Conclusion

This paper proposes a cooperative jamming multi-dimensional joint decision-making algorithm (MA-CJD) based on multi-agent reinforcement learning, addressing a multi-agent electromagnetic countermeasure scenario. A mathematical model was developed to represent the cooperative jamming process, incorporating both radar and jammer models. A Markov game framework was introduced, where jamming targets, models, and power levels were treated as decision variables within a parameterized action space. The QMix algorithm, integrated with the MP-DQN architecture, was applied to enhance learning efficiency and improve decision-making performance.

Compared to existing methods, the MA-CJD algorithm demonstrates significantly improved jamming performance with reduced resource consumption. This approach provides a novel framework for multi-dimensional jamming parameter optimization, validating the effectiveness of combining QMix and MP-DQN in cooperative decision-making tasks. Future work will explore more detailed models and incorporate additional parameters to enhance practical applicability.

Acknowledgements

The authors express their sincere gratitude to Yang Jingfang for offering professional insights and constructive suggestions during the development of the jamming model.

Author contributions

CB designed, implemented, and validated the proposed algorithm. LH developed the modeling approach for the cooperative jamming decision-making scenario. ZN, CM, and YH contributed suggestions for the experimental design. All authors participated in drafting and revising the manuscript and approved the final version for submission.

Funding

The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Data availability

Data will be available when needed.

Declarations

Competing interests

The authors declare no competing interests.

Received: 3 December 2024 Revised: 5 January 2025

Accepted: 16 January 2025 Published online: 05 February 2025

References

- Q. Xing, W. Zhu, Y. Bo, Jamming style selection for small sample radar jamming rule base, in *2018 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC)* (2018), pp. 1–5. <https://doi.org/10.1109/ICSPCC.2018.8567613>
- Q. Xing, W. Zhu, Z. Chi, G. Zheng, Jamming decision under condition of incomplete jamming rule library. *J. Eng.* **2019**(21), 7449–7454 (2019). <https://doi.org/10.1049/joe.2019.0486>
- T. Zheng, G. Xiaoguang, Research on the self-defence electronic jamming decision-making based on the discrete dynamic Bayesian network*. *J. Syst. Eng. Electron.* **19**(4), 702–708 (2008). [https://doi.org/10.1016/S1004-4132\(08\)60142-5](https://doi.org/10.1016/S1004-4132(08)60142-5)
- L. Ye, Research on intelligent communication jamming decision-making method in real-time environment. PhD thesis, Harbin Institute of Technology (2022)
- F. Ye, F. Che, L. Gao, Multiobjective cognitive cooperative jamming decision-making method based on tabu search-artificial bee colony algorithm. *Int. J. Aerosp. Eng.* **2018**(1), 7490895 (2018). <https://doi.org/10.1155/2018/7490895>
- W. Gu, L. Zhu, Y. Bu, W. Yue, X. Cai, Y. Fan, Collaborative jamming decision-making mechanism using ant colony algorithm in electromagnetic antagonism, in *2020 IEEE 20th International Conference on Communication Technology (ICCT)* (2020), pp. 1645–1649. <https://doi.org/10.1109/ICCT50939.2020.9295683>
- D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, P. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis, Mastering the game of go with deep neural networks and tree search. *Nature* **529**(7587), 484–489 (2016). <https://doi.org/10.1038/nature16961>
- D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, D. Hassabis, Mastering the game of go without human knowledge. *Nature* **550**(7676), 354–359 (2017). <https://doi.org/10.1038/nature24270>
- J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, D. Silver, Mastering atari, go, chess and shogi by planning with a learned model. *Nature* **588**(7839), 604–609 (2020). <https://doi.org/10.1038/s41586-020-03051-4>
- OpenAI, B.C. Berner, G. Brockman, B. Chan, V. Cheung, P. D biak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. J zefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, S. Zhang, *Dota 2 with Large Scale Deep Reinforcement Learning* (2019). <https://doi.org/10.48550/arXiv.1912.06680>
- O. Vinyals, I. Babuschkin, W.M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D.H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J.P. Agapiou, M. Jaderberg, A.S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T.L. Payne, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. W nsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, D. Silver, Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature* **575**(7782), 350–354 (2019). <https://doi.org/10.1038/s41586-019-1724-z>
- J. Boron, C. Darken, Developing combat behavior through reinforcement learning in wargames and simulations, in *2020 IEEE Conference on Games (CoG)* (2020), pp. 728–731. <https://doi.org/10.1109/CoG47356.2020.9231609>
- L.-W. Feng, S.-T. Liu, H.-Z. Xu, Multifunctional radar cognitive jamming decision based on dueling double deep q-network. *IEEE Access* **10**, 112150–112157 (2022). <https://doi.org/10.1109/ACCESS.2022.3214842>
- W. Zhang, T. Zhao, Z. Zhao, D. Ma, F. Liu, Performance analysis of deep reinforcement learning-based intelligent cooperative jamming method confronting multi-functional networked radar. *Signal Process.* **207**, 108965 (2023). <https://doi.org/10.1016/j.sigpro.2023.108965>
- Y. Zhang, W. Huo, Y. Huang, C. Zhang, J. Pei, Y. Zhang, J. Yang, Jamming policy generation via heuristic programming reinforcement learning. *IEEE Trans. Aerosp. Electron. Syst.* **59**(6), 8782–8799 (2023). <https://doi.org/10.1109/TAES.2023.3312231>
- W. Zhang, D. Ma, Z. Zhao, F. Liu, Design of cognitive jamming decision-making system against mfr based on reinforcement learning. *IEEE Trans. Veh. Technol.* **72**(8), 10048–10062 (2023). <https://doi.org/10.1109/TVT.2023.3261318>
- Z. Pan, Y. Li, S. Wang, Y. Li, Joint optimization of jamming type selection and power control for countering multifunction radar based on deep reinforcement learning. *IEEE Trans. Aerosp. Electron. Syst.* **59**(4), 4651–4665 (2023). <https://doi.org/10.1109/TAES.2023.3272307>
- M. Sun, S. Yin, D. Zeng, Multi-agent multi-dimensional joint optimisation of jamming decision-making against multi-functional radar. *IET Radar Sonar Navig.* **18**(10), 1980–1990 (2024). <https://doi.org/10.1049/rsn2.12632>
- P. Hernandez-Leal, B. Kartal, M.E. Taylor, A survey and critique of multiagent deep reinforcement learning. *Auton. Agents Multi-Agent Syst.* **33**(6), 750–797 (2019). <https://doi.org/10.1007/s10458-019-09421-1>
- J. Dinneweth, A. Boubezoul, R. Mandiau, S. Espi , Multi-agent reinforcement learning for autonomous vehicles: a survey. *Auton. Intell. Syst.* **2**(1), 27 (2022). <https://doi.org/10.1007/s43684-022-00045-z>
- M. Li, H. Sun, Y. Huang, H. Chen, Shapley value: from cooperative game to explainable artificial intelligence. *Auton. Intell. Syst.* **4**(1), 2 (2024). <https://doi.org/10.1007/s43684-023-00060-8>
- F. Chen, Q. Yu, Multiple unmanned ship coverage and exploration in complex sea areas. *Auton. Intell. Syst.* **4**(1), 14 (2024). <https://doi.org/10.1007/s43684-024-00069-7>
- T. Rashid, M. Samvelyan, C.S. Witt, G. Farquhar, J. Foerster, S. Whiteson, Monotonic value function factorisation for deep multi-agent reinforcement learning. *J. Mach. Learn. Res.* **21**(178), 1–51 (2020)
- R. Lowe, Y. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in *Advances in Neural Information Processing Systems*, vol. 30 (2017), pp. 1–12
- T. Wang, T. Gupta, A. Mahajan, B. Peng, S. Whiteson, C. Zhang, RODE: Learning Roles to Decompose Multi-Agent Tasks (2020). <https://doi.org/10.48550/arXiv.2010.01523>
- C. Feng, X. Fu, Z. Wang, J. Dong, Z. Zhao, T. Pan, An optimization method for collaborative radar anti-jamming based on multi-agent reinforcement learning. *Remote Sens.* **15**(11), 2893 (2023). <https://doi.org/10.3390/rs15112893>
- W. Chen, J. Nie, A maddpg-based multi-agent antagonistic algorithm for sea battlefield confrontation. *Multimed. Syst.* **29**(5), 2991–3000 (2023). <https://doi.org/10.1007/s00530-022-00922-w>
- Z. Gong, Y. Xu, D. Luo, Uav cooperative air combat maneuvering confrontation based on multi-agent reinforcement learning. *Unmanned Syst.* (2022). <https://doi.org/10.1142/S2301385023410029>
- J. Hu, S. Jiang, S.A. Harding, H. Wu, S.-W. Liao, Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning (2023). <https://doi.org/10.48550/arXiv.2102.03479>
- D.W. Tufts, A.J. Cann, On albersheim's detection equation. *IEEE Trans. Aerosp. Electron. Syst.* **AES-19**(4), 643–646 (1983). <https://doi.org/10.1109/TAES.1983.309356>
- R.L. Mitchell, *Radar Signal Simulation* (Artech House, Dedham, 1976)
- S. Zhao, M. Yi, Z. Liu, B. Yang, Deception ecm for multiple jammer sources in distributed multiple-radar systems. *J. Nanjing Univ. Posts Telecommun. Nat. Sci.* **42**(5), 15–20 (2022). <https://doi.org/10.14132/j.cnki.1673-5439.2022.05.003>
- H. Zhou, L. Wang, Z. Guo, Recognition of radar compound jamming based on convolutional neural network. *IEEE Trans. Aerosp. Electron. Syst.* **59**(6), 7380–7394 (2023). <https://doi.org/10.1109/TAES.2023.3288080>
- X. Wang, S. Chen, Y. Zhu, S. Zhang, X. Li, L. Zhu, Application of wavelet scattering network and ensemble learning on deception jamming recognition for ultra-wideband detectors. *IEEE Trans. Microw. Theory Tech.* **72**(4), 2591–2601 (2024). <https://doi.org/10.1109/TMTT.2023.3313872>
- Y. Yang, J. Wang, An overview of multi-agent reinforcement learning from game theoretical perspective (2021). <https://doi.org/10.48550/arXiv.2011.00583>
- P. Sunehag, G. Lever, A. Gruslys, W.M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J.Z. Leibo, K. Tuyls, T. Graepel, Value-decomposition networks for cooperative multi-agent learning (2017). <https://doi.org/10.48550/arXiv.1706.05296>
- H. Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning. *Proc. AAAI Conf. Artif. Intell.* **30**(1) (2016). <https://doi.org/10.1609/aaai.v30i1.10295>
- K. Son, D. Kim, W.J. Kang, D.E. Hostallero, Y. Yi, Qtran: learning to factorize with transformation for cooperative multi-agent reinforcement learning, in *Proceedings of the 36th International Conference on Machine Learning* (2019), pp. 5887–5896

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Submit your manuscript to a SpringerOpen[®] journal and benefit from:

- Convenient online submission
- Rigorous peer review
- Open access: articles freely available online
- High visibility within the field
- Retaining the copyright to your article

Submit your next manuscript at ► [springeropen.com](https://www.springeropen.com)